

Computer Science I

Week 2

Tutorial Session

Question 1 - Identifiers

Which of the following are valid C identifiers?

count	2count	total_sum	total-sum	_value	value2
-------	--------	-----------	-----------	--------	--------

For each invalid identifier, explain which naming rule it breaks.

Question 2 - Constants And Assignment

Consider the following code:

```
const int limit = 10;
int value = 4;

value = value + 2;
limit = 12;
```

Which assignments are valid? What are the final values of `value` and `limit` if the invalid line is removed?

Question 3 - Integer And Floating-Point Division

Determine the value and type of each variable:

```
int a = 7 / 2;
double b = 7 / 2;
double c = 7.0 / 2;
double d = (double) 7 / 2;
```

Explain why assigning a result to a `double` does not necessarily make the division floating-point division.

Question 4 - Division And Remainder

Determine the values of `hours`, `minutes`, and `seconds`:

```
int total_seconds = 7384;
int hours = total_seconds / 3600;
int minutes = (total_seconds % 3600) / 60;
int seconds = total_seconds % 60;
```

Briefly explain the role of the remainder operator `%`.

Question 5 - Comparisons And Logical Operators

Determine the value of each variable. Recall that a comparison or logical expression in C evaluates to either 0 or 1.

```
int x = 5;
int a = x > 2;
int b = x < 3;
int c = (x > 2) && (x < 10);
int d = (x == 5) || (x == 8);
int e = !(x == 5);
```

Question 6 - Tracing A Loop

Trace the values of `i` and `sum` after each iteration. What is printed at the end?

```
int i;
int sum = 0;

for (i = 1; i <= 4; i = i + 1) {
    sum += i;
}

printf("%d\n", sum);
```

Rewrite the loop using `while` instead of `for`.

Bonus Question 7 - Precedence And Types

Without running the code, determine the value and type of each variable:

```
int a = 2 + 3 * 4;
int b = (2 + 3) * 4;
int c = 17 % 5 * 2;
double d = 5 / 2 + 0.5;
```

Explain the order in which the operations are performed.

Bonus Question 8 - Short-Circuit Evaluation

Consider the following code:

```
int x = 0;
int a = (x != 0) && (10 / x > 1);
int b = (x == 0) || (10 / x > 1);
```

Determine the values of `a` and `b`. Explain why neither line attempts to divide by zero. Would it be safe to reverse the two operands of either logical expression?

Coding Exercise 1 - Ceiling Division

Write a function:

```
int ceil_div(int a, int b);
```

Assume that $a \geq 0$ and $b > 0$. The function must return the smallest integer that is greater than or equal to the mathematical value of a / b .

Examples:

```
ceil_div(13, 5) returns 3
ceil_div(15, 5) returns 3
ceil_div(0, 5)  returns 0
```

Write a complete program that defines the function and calls it from `main` for the three examples above.

Coding Exercise 2 - Clamp A Value

Write a function:

```
int clamp(int x, int lower, int upper);
```

Assume that $lower \leq upper$. The function must:

- return `lower` when $x < lower$;
- return `upper` when $x > upper$;
- return x otherwise.

For example, with the interval $[-2, 4]$:

```
clamp(-5, -2, 4) returns -2
clamp(3, -2, 4)  returns 3
clamp(10, -2, 4) returns 4
```

Write a complete program that defines the function and calls it from `main` for the three examples above.

Coding Exercise 3 - Detect Integer Overflow

The header `limits.h` defines the minimum and maximum values of integer types. For example:

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("Smallest int: %d\n", INT_MIN);
    printf("Largest int:  %d\n", INT_MAX);

    return 0;
}
```

If the mathematical result of an operation is outside this range, a signed integer overflow occurs. Signed integer overflow has undefined behavior in C, so the overflowing addition must not be performed, even temporarily.

Write a function:

```
int addition_is_safe(int a, int b);
```

The function must return 1 if $a + b$ can be represented by an `int`, and 0 otherwise. It must make this decision before evaluating $a + b$.

Write a complete program that tests at least these cases:

```
addition_is_safe(100, -50) returns 1
addition_is_safe(INT_MAX, 1) returns 0
addition_is_safe(INT_MIN, -1) returns 0
```